02 — Análisis exploratorio (EDA)

Give Me Some Credit — Análisis de riesgo crediticio

Autor: Mario Zarza Zárraga
Entrada: `data/cs-training-limpio.csv` (salida de `01_limpieza`)
Objetivo: entender qué variables se asocian más con la mora grave, probar la hipótesis principal y dejar hallazgos accionables para el modelo (`03_modelo`).

Hipótesis: *«Los atrasos pasados son lo que más pesa»*.

0. Librerías, estilo y carga

In [1]:
```
import warnings
from pathlib import Path

import matplotlib.pyplot as plt
import numpy as np
import pandas as pd
import seaborn as sns
from sklearn.metrics import roc_auc_score

warnings.filterwarnings("ignore", category=FutureWarning)

ROOT = Path("..")
DATA = ROOT / "data"
FIG = ROOT / "reports" / "figures"
FIG.mkdir(parents=True, exist_ok=True)

sns.set_theme(style="whitegrid", context="notebook")
plt.rcParams.update({
    "figure.figsize": (9, 5),
    "axes.titlesize": 13,
    "axes.labelsize": 11,
    "axes.titleweight": "bold",
    "font.size": 10,
})

COLOR_MORA = "#c0392b"
COLOR_OK = "#2980b9"
```

In [2]:
```
df = pd.read_csv(DATA / "cs-training-limpio.csv")
print(f"Forma: {df.shape[0]:,} filas × {df.shape[1]} columnas")
print(f"Columnas: {list(df.columns)}")
df.head()
```

```
Forma: 149,999 filas × 15 columnas
Columnas: ['mora_grave', 'uso_credito_revolvente', 'edad', 'atrasos_30_59',
'ratio_deuda', 'ingreso_mensual', 'lineas_credito_abiertas', 'atrasos_90',
'prestamos_hipotecarios', 'atrasos_60_89', 'dependientes', 'codigo_atraso_es
pecial', 'ingreso_faltante', 'ratio_deuda_dudoso', 'uso_revolvente_gt1']
```

Out[2]:

	mora_grave	uso_credito_revolvente	edad	atrasos_30_59	ratio_deuda	ingreso_men
0	1	0.766127	45	2.0	0.802982	91
1	0	0.957151	40	0.0	0.121876	26
2	0	0.658180	38	1.0	0.085113	30
3	0	0.233810	30	0.0	0.036050	33
4	0	0.907239	49	1.0	0.024926	635

1. Balance de la variable objetivo

`mora_grave = 1` si el cliente cayó en mora ≥90 días en los siguientes 2 años.

In [3]:

```
tasa = df["mora_grave"].mean()
conteo = df["mora_grave"].value_counts().sort_index()
print(f"Tasa de mora grave: {tasa:.2%} ({int(conteo.get(1, 0)):,} de {len(df
print(conteo.rename({0: "sin mora (0)", 1: "mora grave (1)"}))

fig, ax = plt.subplots(figsize=(7, 4.5))
bars = ax.bar(
    ["Sin mora", "Mora grave"],
    [conteo[0], conteo[1]],
    color=[COLOR_OK, COLOR_MORA],
    width=0.55,
)
ax.set_title("Balance de la variable objetivo (mora_grave)")
ax.set_ylabel("Número de clientes")
for b, n_ in zip(bars, [conteo[0], conteo[1]]):
    ax.text(
        b.get_x() + b.get_width() / 2,
        b.get_height(),
        f"{n_:,}\n({n_ / len(df):.1%})",
        ha="center",
        va="bottom",
        fontsize=10,
    )
ax.set_ylim(0, conteo.max() * 1.15)
fig.tight_layout()
fig.savefig(FIG / "01_balance_objetivo.png", dpi=140, bbox_inches="tight")
plt.show()
```

```
Tasa de mora grave: 6.68% (10,026 de 149,999)
mora_grave
sin mora (0)      139973
mora grave (1)     10026
Name: count, dtype: int64
```

```
/tmp/ipykernel_76472/2472109396.py:27: UserWarning: FigureCanvasAgg is non-i
nteractive, and thus cannot be shown
  plt.show()
```

> El dataset está **desbalanceado** (~6.7% de mora). En el modelado habrá que cuidar métricas (AUC, recall de la clase positiva) y, si aplica, ponderación o muestreo.

2. Hipótesis: ¿pesan más los atrasos pasados?

Creamos `total_atrasos` = suma de las tres columnas de atraso (ya limpias) y vemos la **tasa de mora** por:

- cada columna de atraso (agrupada en 0, 1, 2, 3+)
- el total combinado
- la bandera `codigo_atraso_especial` (códigos 96/98 del sistema)

In [4]:
```
df["total_atrasos"] = (
    df["atrasos_30_59"] + df["atrasos_60_89"] + df["atrasos_90"]
).astype(int)


def bucket_atraso(s: pd.Series) -> pd.Series:
    # Agrupa conteos de atraso en 0, 1, 2, 3+
    out = s.clip(upper=3).astype(int).astype(str).replace({"3": "3+"})
    return pd.Categorical(out, categories=["0", "1", "2", "3+"], ordered=Tru


def tasa_por_grupo(data: pd.DataFrame, col: str) -> pd.DataFrame:
    g = data.groupby(col, observed=True)["mora_grave"].agg(["mean", "count"]
    g = g.rename(columns={"mean": "tasa_mora", "count": "n"})
    g["tasa_mora_pct"] = (g["tasa_mora"] * 100).round(2)
    return g


for col in ["atrasos_30_59", "atrasos_60_89", "atrasos_90", "total_atrasos"]
    bcol = f"{col}_bucket"
    df[bcol] = bucket_atraso(df[col])
    tabla = tasa_por_grupo(df, bcol)
    print(f"\n=== {col} ===")
    print(tabla)
```

```
=== atrasos_30_59 ===
                       tasa_mora       n  tasa_mora_pct
atrasos_30_59_bucket
0                       0.041081  126287           4.11
1                       0.150262   16032          15.03
2                       0.265115    4598          26.51
3+                      0.392602    3082          39.26
=== atrasos_60_89 ===
                       tasa_mora       n  tasa_mora_pct
atrasos_60_89_bucket
0                       0.051891  142664           5.19
1                       0.310068    5731          31.01
2                       0.501789    1118          50.18
3+                      0.586420     486          58.64

=== atrasos_90 ===
                     tasa_mora       n  tasa_mora_pct
atrasos_90_bucket
0                     0.047213  141930           4.72
1                     0.336639    5243          33.66
2                     0.499035    1555          49.90
3+                    0.616837    1271          61.68

=== total_atrasos ===
                       tasa_mora       n  tasa_mora_pct
total_atrasos_bucket
0                       0.028447  119906           2.84
1                       0.122144   17242          12.21
2                       0.240828    5942          24.08
3+                      0.445506    6909          44.55
```

In [5]:
```
fig, axes = plt.subplots(1, 4, figsize=(14, 4.2), sharey=True)
pairs = [
    ("atrasos_30_59_bucket", "Atrasos 30–59 días"),
    ("atrasos_60_89_bucket", "Atrasos 60–89 días"),
    ("atrasos_90_bucket", "Atrasos 90+ días"),
    ("total_atrasos_bucket", "Total atrasos"),
]
for ax, (col, titulo) in zip(axes, pairs):
    t = tasa_por_grupo(df, col)
    ax.bar(t.index.astype(str), t["tasa_mora"] * 100, color=COLOR_MORA, alph
    ax.set_title(titulo, fontsize=11)
    ax.set_xlabel("Núm. de eventos")
    for i, (_, row) in enumerate(t.iterrows()):
        ax.text(
            i,
            row["tasa_mora"] * 100 + 0.8,
            f"{row['tasa_mora'] * 100:.1f}%",
            ha="center",
            va="bottom",
            fontsize=8,
        )
axes[0].set_ylabel("Tasa de mora (%)")
fig.suptitle("Tasa de mora según historial de atrasos", fontweight="bold", y
fig.tight_layout()
```

```
fig.savefig(FIG / "02_mora_por_atrasos.png", dpi=140, bbox_inches="tight")
plt.show()
```

```
/tmp/ipykernel_76472/2837367215.py:26: UserWarning: FigureCanvasAgg is non-i
nteractive, and thus cannot be shown
  plt.show()
```

In [6]:
```
tab_codigo = tasa_por_grupo(df, "codigo_atraso_especial")
tab_codigo.index = tab_codigo.index.map({0: "Sin código especial", 1: "Códig
print(tab_codigo)

fig, ax = plt.subplots(figsize=(6.5, 4.2))
ax.bar(
    tab_codigo.index.astype(str),
    tab_codigo["tasa_mora"] * 100,
    color=[COLOR_OK, COLOR_MORA],
    width=0.5,
)
ax.set_title("Tasa de mora: código de atraso especial (96/98)")
ax.set_ylabel("Tasa de mora (%)")
for i, (_, row) in enumerate(tab_codigo.iterrows()):
    ax.text(
        i,
        row["tasa_mora"] * 100 + 1.5,
        f"{row['tasa_mora'] * 100:.1f}%\n(n={int(row['n']):,})",
        ha="center",
        va="bottom",
        fontsize=9,
    )
ax.set_ylim(0, max(tab_codigo["tasa_mora"] * 100) * 1.25)
fig.tight_layout()
fig.savefig(FIG / "03_mora_codigo_especial.png", dpi=140, bbox_inches="tight
plt.show()
```

```
                        tasa_mora       n  tasa_mora_pct
codigo_atraso_especial
Sin código especial      0.065979  149730           6.60
Código 96/98             0.546468     269          54.65
```

```
/tmp/ipykernel_76472/3701141980.py:26: UserWarning: FigureCanvasAgg is non-i
nteractive, and thus cannot be shown
  plt.show()
```

3. Tasa de mora por otras variables

Segmentamos edad, ingreso, uso revolvente, ratio de deuda, líneas abiertas, hipotecas y dependientes.

In [7]:
```
bins_edad = [0, 30, 40, 50, 60, 70, 120]
labels_edad = ["≤30", "31–40", "41–50", "51–60", "61–70", "70+"]
df["edad_banda"] = pd.cut(df["edad"], bins=bins_edad, labels=labels_edad, ri
tab_edad = tasa_por_grupo(df, "edad_banda")
print("=== Edad ===")
print(tab_edad)
```

```
fig, ax = plt.subplots(figsize=(8, 4.2))
ax.bar(tab_edad.index.astype(str), tab_edad["tasa_mora"] * 100, color="#8e44
ax.set_title("Tasa de mora por banda de edad")
ax.set_xlabel("Edad")
ax.set_ylabel("Tasa de mora (%)")
for i, (_, row) in enumerate(tab_edad.iterrows()):
    ax.text(
        i,
        row["tasa_mora"] * 100 + 0.2,
        f"{row['tasa_mora'] * 100:.1f}%",
        ha="center",
        va="bottom",
        fontsize=9,
    )
fig.tight_layout()
fig.savefig(FIG / "04_mora_por_edad.png", dpi=140, bbox_inches="tight")
plt.show()
```

```
=== Edad ===
            tasa_mora      n  tasa_mora_pct
edad_banda
≤30          0.115646  10757          11.56
31–40        0.098196  24339           9.82
41–50        0.082570  35037           8.26
51–60        0.061742  34806           6.17
61–70        0.034714  27424           3.47
70+          0.022567  17636           2.26
```

```
/tmp/ipykernel_76472/66466576.py:24: UserWarning: FigureCanvasAgg is non-int
eractive, and thus cannot be shown
  plt.show()
```

In [8]:

```
con_ing = df["ingreso_faltante"] == 0
df.loc[con_ing, "ingreso_q"] = pd.qcut(
    df.loc[con_ing, "ingreso_mensual"],
    q=5,
    labels=["Q1 (bajo)", "Q2", "Q3", "Q4", "Q5 (alto)"],
)
df["ingreso_grupo"] = df["ingreso_q"].astype(object)
df.loc[df["ingreso_faltante"] == 1, "ingreso_grupo"] = "Sin ingreso reportac
orden_ing = ["Q1 (bajo)", "Q2", "Q3", "Q4", "Q5 (alto)", "Sin ingreso report
df["ingreso_grupo"] = pd.Categorical(df["ingreso_grupo"], categories=orden_i

tab_ing = tasa_por_grupo(df, "ingreso_grupo")
print("=== Ingreso (quintiles + faltantes) ===")
print(tab_ing)

fig, ax = plt.subplots(figsize=(9, 4.2))
cols_bar = [
    COLOR_MORA if i == 0 else ("#7f8c8d" if i == 5 else COLOR_OK)
    for i in range(len(tab_ing))
]
ax.bar(tab_ing.index.astype(str), tab_ing["tasa_mora"] * 100, color=cols_bar
ax.set_title("Tasa de mora por quintil de ingreso (+ sin ingreso)")
ax.set_ylabel("Tasa de mora (%)")
ax.tick_params(axis="x", rotation=15)
for i, (_, row) in enumerate(tab_ing.iterrows()):
```

```
    ax.text(
        i,
        row["tasa_mora"] * 100 + 0.15,
        f"{row['tasa_mora'] * 100:.1f}%",
        ha="center",
        va="bottom",
        fontsize=8,
    )
fig.tight_layout()
fig.savefig(FIG / "05_mora_por_ingreso.png", dpi=140, bbox_inches="tight")
plt.show()
```

```
=== Ingreso (quintiles + faltantes) ===
                      tasa_mora      n  tasa_mora_pct
ingreso_grupo
Q1 (bajo)              0.090775  25084           9.08
Q2                     0.085954  23024           8.60
Q3                     0.070128  24056           7.01
Q4                     0.055306  24084           5.53
Q5 (alto)              0.045046  24020           4.50
Sin ingreso reportado  0.056137  29731           5.61
```

```
/tmp/ipykernel_76472/3953877021.py:36: UserWarning: FigureCanvasAgg is non-i
nteractive, and thus cannot be shown
  plt.show()
```

In [9]:
```
# CSV limpio capado en p99 (~1.09); la banda ">1" captura ese remanente
bins_uso = [-0.01, 0.3, 0.6, 0.9, 1.0, 100]
labels_uso = ["0–0.3", "0.3–0.6", "0.6–0.9", "0.9–1", ">1"]
df["uso_banda"] = pd.cut(df["uso_credito_revolvente"], bins=bins_uso, labels
tab_uso = tasa_por_grupo(df, "uso_banda")
print("=== Uso revolvente ===")
print(tab_uso)

fig, ax = plt.subplots(figsize=(8, 4.2))
ax.bar(tab_uso.index.astype(str), tab_uso["tasa_mora"] * 100, color="#d35400
ax.set_title("Tasa de mora por uso de crédito revolvente")
ax.set_xlabel("Uso revolvente")
ax.set_ylabel("Tasa de mora (%)")
for i, (_, row) in enumerate(tab_uso.iterrows()):
    ax.text(
        i,
        row["tasa_mora"] * 100 + 0.3,
        f"{row['tasa_mora'] * 100:.1f}%",
        ha="center",
        va="bottom",
        fontsize=9,
    )
fig.tight_layout()
fig.savefig(FIG / "06_mora_por_uso_revolvente.png", dpi=140, bbox_inches="ti
plt.show()
```

```
=== Uso revolvente ===
           tasa_mora      n  tasa_mora_pct
uso_banda
0–0.3       0.022189  92882           2.22
0.3–0.6     0.066798  21887           6.68
0.6–0.9     0.133030  15162          13.30
0.9–1       0.194005  16747          19.40
>1          0.372478   3321          37.25
```

```
/tmp/ipykernel_76472/2087307577.py:25: UserWarning: FigureCanvasAgg is non-i
nteractive, and thus cannot be shown
  plt.show()
```

In [10]:
```
df_ratio = df[df["ratio_deuda_dudoso"] == 0].copy()
bins_ratio = [-0.01, 0.2, 0.4, 0.6, 1.0, float(df_ratio["ratio_deuda"].max()
labels_ratio = ["0–0.2", "0.2–0.4", "0.4–0.6", "0.6–1", ">1"]
df_ratio["ratio_banda"] = pd.cut(df_ratio["ratio_deuda"], bins=bins_ratio, l
tab_ratio = tasa_por_grupo(df_ratio, "ratio_banda")
print("=== Ratio de deuda (solo ingreso reportado) ===")
print(tab_ratio)
n_dudoso = int(df["ratio_deuda_dudoso"].sum())
tasa_dudoso = df.loc[df["ratio_deuda_dudoso"] == 1, "mora_grave"].mean()
print(f"\nTasa mora si ratio_deuda_dudoso=1: {tasa_dudoso:.2%} (n={n_dudoso:

fig, ax = plt.subplots(figsize=(8, 4.2))
ax.bar(tab_ratio.index.astype(str), tab_ratio["tasa_mora"] * 100, color="#16
ax.set_title("Tasa de mora por ratio de deuda (ingreso reportado)")
ax.set_xlabel("Ratio de deuda")
ax.set_ylabel("Tasa de mora (%)")
for i, (_, row) in enumerate(tab_ratio.iterrows()):
    ax.text(
        i,
        row["tasa_mora"] * 100 + 0.15,
        f"{row['tasa_mora'] * 100:.1f}%",
        ha="center",
        va="bottom",
        fontsize=9,
    )
fig.tight_layout()
fig.savefig(FIG / "07_mora_por_ratio_deuda.png", dpi=140, bbox_inches="tight
plt.show()
```

```
=== Ratio de deuda (solo ingreso reportado) ===
             tasa_mora      n  tasa_mora_pct
ratio_banda
0–0.2         0.060885  40667           6.09
0.2–0.4       0.056316  38284           5.63
0.4–0.6       0.076948  21521           7.69
0.6–1         0.106344  12563          10.63
>1            0.101341   7233          10.13

Tasa mora si ratio_deuda_dudoso=1: 5.61% (n=29,731)
```

```
/tmp/ipykernel_76472/4151808311.py:28: UserWarning: FigureCanvasAgg is non-i
nteractive, and thus cannot be shown
  plt.show()
```

In [11]:
```
fig, axes = plt.subplots(1, 3, figsize=(13, 4.2), sharey=True)
specs = [
    (
        "lineas_credito_abiertas",
        "Líneas de crédito abiertas",
        pd.cut(
            df["lineas_credito_abiertas"],
            bins=[-0.1, 2, 5, 8, 12, 10_000],
            labels=["0–2", "3–5", "6–8", "9–12", "13+"],
        ),
    ),
    (
        "prestamos_hipotecarios",
        "Préstamos hipotecarios",
        pd.cut(
            df["prestamos_hipotecarios"],
            bins=[-0.1, 0.5, 1.5, 2.5, 10_000],
            labels=["0", "1", "2", "3+"],
        ),
    ),
    (
        "dependientes",
        "Dependientes",
        pd.cut(
            df["dependientes"],
            bins=[-0.1, 0.5, 1.5, 2.5, 3.5, 10_000],
            labels=["0", "1", "2", "3", "4+"],
        ),
    ),
]
for ax, (nombre, titulo, serie) in zip(axes, specs):
    tmp = df.copy()
    tmp["_b"] = serie
    t = tasa_por_grupo(tmp, "_b")
    print(f"\n=== {nombre} ===")
    print(t)
    ax.bar(t.index.astype(str), t["tasa_mora"] * 100, color="#2c3e50", alpha
    ax.set_title(titulo, fontsize=11)
    ax.set_xlabel(nombre)
    for i, (_, row) in enumerate(t.iterrows()):
        ax.text(
            i,
            row["tasa_mora"] * 100 + 0.15,
            f"{row['tasa_mora'] * 100:.1f}%",
            ha="center",
            va="bottom",
            fontsize=8,
        )
axes[0].set_ylabel("Tasa de mora (%)")
fig.suptitle("Tasa de mora por líneas, hipotecas y dependientes", fontweight
fig.tight_layout()
fig.savefig(FIG / "08_mora_lineas_hipotecas_deps.png", dpi=140, bbox_inches=
plt.show()
```

```
=== lineas_credito_abiertas ===
      tasa_mora      n  tasa_mora_pct
_b
0–2    0.129156  12992          12.92
3–5    0.066790  33598           6.68
6–8    0.053450  39420           5.35
9–12   0.059248  36305           5.92
13+    0.066681  27684           6.67

=== prestamos_hipotecarios ===
    tasa_mora      n  tasa_mora_pct
_b
0    0.083149  56188           8.31
1    0.052505  52338           5.25
2    0.055994  31521           5.60
3+   0.084506   9952           8.45

=== dependientes ===
    tasa_mora      n  tasa_mora_pct
_b
0    0.058067  90826           5.81
1    0.073529  26316           7.35
2    0.081143  19521           8.11
3    0.088263   9483           8.83
4+   0.102777   3853          10.28
```

```
/tmp/ipykernel_76472/416681506.py:53: UserWarning: FigureCanvasAgg is non-in
teractive, and thus cannot be shown
  plt.show()
```

4. Ranking de poder predictivo: IV (WoE) y AUC univariado

En credit scoring se usa mucho el **Information Value (IV)** con **Weight of Evidence (WoE)**:

- Se parte la variable en bins.
- En cada bin se calcula WoE = ln(% buenos / % malos), donde “malo” = cayó en mora.
- IV = Σ (%buenos − %malos) × WoE.

Regla práctica (Siddiqi): IV < 0.02 inútil; 0.02–0.1 débil; 0.1–0.3 medio; 0.3–0.5 fuerte; > 0.5 muy fuerte / sospechosamente fuerte.

Complementamos con **AUC univariado** (qué tan bien ordena la variable a los clientes por riesgo, sola).

In [12]:
```
def woe_iv(data, feature, target="mora_grave", n_bins=10, categorical=False)
    # Information Value con bins WoE
    d = data[[feature, target]].copy().dropna()
    if d[feature].nunique() <= 1:
        return 0.0
```

```
    if categorical or d[feature].nunique() <= n_bins:
        d["bin"] = d[feature].astype(str)
    else:
        try:
            d["bin"] = pd.qcut(
                d[feature], q=min(n_bins, d[feature].nunique()), duplicates=
            )
        except ValueError:
            d["bin"] = pd.cut(d[feature], bins=min(n_bins, d[feature].nuniqu

    total_good = (d[target] == 0).sum()
    total_bad = (d[target] == 1).sum()
    if total_good == 0 or total_bad == 0:
        return 0.0

    g = d.groupby("bin", observed=True)[target].agg(n="count", bads="sum")
    g["goods"] = g["n"] - g["bads"]
    g["pct_good"] = g["goods"].clip(lower=0.5) / total_good
    g["pct_bad"] = g["bads"].clip(lower=0.5) / total_bad
    g["woe"] = np.log(g["pct_good"] / g["pct_bad"])
    g["iv"] = (g["pct_good"] - g["pct_bad"]) * g["woe"]
    return float(g["iv"].sum())


def univariate_auc(data, feature, target="mora_grave"):
    # max(auc, 1-auc) para que la dirección no importe
    d = data[[feature, target]].dropna()
    if d[feature].nunique() <= 1:
        return 0.5
    try:
        auc = roc_auc_score(d[target], d[feature])
    except ValueError:
        return 0.5
    return float(max(auc, 1 - auc))


features_iv = [
    "atrasos_30_59",
    "atrasos_60_89",
    "atrasos_90",
    "total_atrasos",
    "codigo_atraso_especial",
    "uso_credito_revolvente",
    "uso_revolvente_gt1",
    "edad",
    "ingreso_mensual",
    "ingreso_faltante",
    "ratio_deuda",
    "ratio_deuda_dudoso",
    "lineas_credito_abiertas",
    "prestamos_hipotecarios",
    "dependientes",
]
flags = {
    "codigo_atraso_especial",
```

```
    "ingreso_faltante",
    "ratio_deuda_dudoso",
    "uso_revolvente_gt1",
}

# Para atrasos (muy sesgados a 0), IV con valores crudos + qcut colapsa a 1
# Usamos los mismos buckets del EDA (0/1/2/3+) para medir su poder real.
atraso_raw = {"atrasos_30_59", "atrasos_60_89", "atrasos_90", "total_atrasos

rows = []
for f in features_iv:
    if f in atraso_raw:
        tmp = df.copy()
        tmp["_iv_bin"] = bucket_atraso(tmp[f]).astype(str)
        iv = woe_iv(tmp, "_iv_bin", categorical=True)
    else:
        iv = woe_iv(df, f, categorical=(f in flags or df[f].nunique() <= 8))
    auc = univariate_auc(df, f)
    rows.append({"variable": f, "IV": iv, "AUC_univ": auc})

ranking = pd.DataFrame(rows).sort_values("IV", ascending=False).reset_index(
ranking["IV"] = ranking["IV"].round(4)
ranking["AUC_univ"] = ranking["AUC_univ"].round(4)


def iv_label(v):
    if v < 0.02:
        return "inútil"
    if v < 0.1:
        return "débil"
    if v < 0.3:
        return "medio"
    if v < 0.5:
        return "fuerte"
    return "muy fuerte"


ranking["fuerza_IV"] = ranking["IV"].map(iv_label)
print("Ranking por Information Value (y AUC univariado):")
ranking
```

```
Ranking por Information Value (y AUC univariado):
```

Out[12]:

	variable	IV	AUC_univ	fuerza_IV
0	total_atrasos	1.3483	0.7680	muy fuerte
1	uso_credito_revolvente	1.1134	0.7778	muy fuerte
2	atrasos_90	0.8274	0.6501	muy fuerte
3	atrasos_30_59	0.7015	0.6819	muy fuerte
4	atrasos_60_89	0.5529	0.6147	muy fuerte
5	edad	0.2592	0.6353	medio
6	uso_revolvente_gt1	0.2421	0.5542	medio
7	ratio_deuda	0.0737	0.5238	débil
8	ingreso_mensual	0.0670	0.5710	débil
9	lineas_credito_abiertas	0.0669	0.5445	débil
10	codigo_atraso_especial	0.0391	0.5069	débil
11	dependientes	0.0250	0.5476	débil
12	prestamos_hipotecarios	0.0121	0.5373	inútil
13	ingreso_faltante	0.0077	0.5170	inútil
14	ratio_deuda_dudoso	0.0077	0.5170	inútil

In [13]:
```
fig, ax = plt.subplots(figsize=(9, 6.5))
plot_df = ranking.sort_values("IV", ascending=True)
colors = plot_df["IV"].apply(
    lambda v: "#c0392b"
    if v >= 0.3
    else ("#e67e22" if v >= 0.1 else ("#3498db" if v >= 0.02 else "#95a5a6")
)
ax.barh(plot_df["variable"], plot_df["IV"], color=colors)
ax.axvline(0.02, color="gray", ls="--", lw=0.8, label="IV 0.02 (débil)")
ax.axvline(0.1, color="orange", ls="--", lw=0.8, label="IV 0.1 (medio)")
ax.axvline(0.3, color="red", ls="--", lw=0.8, label="IV 0.3 (fuerte)")
ax.set_xlabel("Information Value (IV)")
ax.set_title("Poder predictivo univariado (IV) — ¿qué pesa más?")
ax.legend(loc="lower right", fontsize=8)
for y, (iv, auc) in enumerate(zip(plot_df["IV"], plot_df["AUC_univ"])):
    ax.text(iv + 0.005, y, f"IV={iv:.3f} | AUC={auc:.3f}", va="center", font
fig.tight_layout()
fig.savefig(FIG / "09_ranking_IV_AUC.png", dpi=140, bbox_inches="tight")
plt.show()

print("\nTop 5 por IV:")
print(ranking.head(5).to_string(index=False))
print("\nTop 5 por AUC univariado:")
print(ranking.sort_values("AUC_univ", ascending=False).head(5).to_string(inc
```

```
Top 5 por IV:
              variable     IV  AUC_univ  fuerza_IV
         total_atrasos 1.3483    0.7680 muy fuerte
uso_credito_revolvente 1.1134    0.7778 muy fuerte
            atrasos_90 0.8274    0.6501 muy fuerte
         atrasos_30_59 0.7015    0.6819 muy fuerte
         atrasos_60_89 0.5529    0.6147 muy fuerte

Top 5 por AUC univariado:
              variable     IV  AUC_univ  fuerza_IV
uso_credito_revolvente 1.1134    0.7778 muy fuerte
         total_atrasos 1.3483    0.7680 muy fuerte
         atrasos_30_59 0.7015    0.6819 muy fuerte
            atrasos_90 0.8274    0.6501 muy fuerte
                 edad 0.2592    0.6353      medio
/tmp/ipykernel_76472/3814160002.py:19: UserWarning: FigureCanvasAgg is non-i
nteractive, and thus cannot be shown
  plt.show()
```

In [14]:
```
top_iv = ranking.iloc[0]["variable"]
atraso_vars = {
    "atrasos_30_59",
    "atrasos_60_89",
    "atrasos_90",
    "total_atrasos",
    "codigo_atraso_especial",
}
top3 = set(ranking.head(3)["variable"])
n_atraso_top3 = len(top3 & atraso_vars)
hipotesis_ok = (top_iv in atraso_vars) or (n_atraso_top3 >= 2)

print("=== Veredicto hipótesis ===")
print(f"Variable #1 por IV: {top_iv} (IV={ranking.iloc[0]['IV']:.4f})")
print(f"Atraso-related en top 3: {sorted(top3 & atraso_vars)}")
print(f"¿Hipótesis se sostiene? {'SÍ' if hipotesis_ok else 'NO / PARCIAL'}")
uso_iv = ranking.loc[ranking["variable"] == "uso_credito_revolvente", "IV"].
tot_iv = ranking.loc[ranking["variable"] == "total_atrasos", "IV"].values[0]
a90_iv = ranking.loc[ranking["variable"] == "atrasos_90", "IV"].values[0]
print(f"IV total_atrasos={tot_iv:.4f} | atrasos_90={a90_iv:.4f} | uso_revolv
```

```
=== Veredicto hipótesis ===
Variable #1 por IV: total_atrasos (IV=1.3483)
Atraso-related en top 3: ['atrasos_90', 'total_atrasos']
¿Hipótesis se sostiene? SÍ
IV total_atrasos=1.3483 | atrasos_90=0.8274 | uso_revolvente=1.1134
```

5. Correlaciones y multicolinealidad

In [15]:
```
num_cols = [
    "mora_grave",
    "uso_credito_revolvente",
    "edad",
    "atrasos_30_59",
    "atrasos_60_89",
    "atrasos_90",
```

```
    "total_atrasos",
    "ratio_deuda",
    "ingreso_mensual",
    "lineas_credito_abiertas",
    "prestamos_hipotecarios",
    "dependientes",
    "codigo_atraso_especial",
    "ingreso_faltante",
    "uso_revolvente_gt1",
]
corr = df[num_cols].corr()

fig, ax = plt.subplots(figsize=(11, 9))
sns.heatmap(
    corr,
    ax=ax,
    cmap="RdBu_r",
    center=0,
    vmin=-1,
    vmax=1,
    annot=True,
    fmt=".2f",
    annot_kws={"size": 7},
    square=True,
    cbar_kws={"shrink": 0.75},
)
ax.set_title("Mapa de correlaciones (Pearson)")
fig.tight_layout()
fig.savefig(FIG / "10_correlacion_heatmap.png", dpi=140, bbox_inches="tight"
plt.show()

atraso_corr = df[
    ["atrasos_30_59", "atrasos_60_89", "atrasos_90", "total_atrasos", "mora_
].corr()
print("Correlación entre columnas de atraso:")
print(atraso_corr.round(3))
```

```
Correlación entre columnas de atraso:
               atrasos_30_59  atrasos_60_89  atrasos_90  total_atrasos  \
atrasos_30_59          1.000          0.306       0.218          0.821
atrasos_60_89          0.306          1.000       0.295          0.623
atrasos_90             0.218          0.295       1.000          0.667
total_atrasos          0.821          0.623       0.667          1.000
mora_grave             0.271          0.266       0.312          0.389

               mora_grave
atrasos_30_59       0.271
atrasos_60_89       0.266
atrasos_90          0.312
total_atrasos       0.389
mora_grave          1.000
```

```
/tmp/ipykernel_76472/3005761056.py:37: UserWarning: FigureCanvasAgg is non-i
nteractive, and thus cannot be shown
  plt.show()
```

Nota sobre multicolinealidad

Las tres columnas de atraso suelen moverse juntas (quien atrasa 90+ casi siempre atrasa también 30–59). Eso **infla coeficientes** en una regresión logística si entran las tres crudas.

Implicación para `03_modelo`:

- Preferir `total_atrasos` + `codigo_atraso_especial`, **o**
- Usar un modelo de árboles (menos sensible a colinealidad), **o**
- Quedarse con la de mayor IV y la bandera especial.

6. Hallazgos clave (para un gerente de crédito / fintech)

Los números exactos salen de las celdas anteriores; aquí va la lectura de negocio.

In [16]:
```
r = ranking.set_index("variable")

tasa_global = df["mora_grave"].mean()
t0 = df.loc[df["total_atrasos"] == 0, "mora_grave"].mean()
t3 = df.loc[df["total_atrasos"] >= 3, "mora_grave"].mean()
tc = df.loc[df["codigo_atraso_especial"] == 1, "mora_grave"].mean()
uso_bajo = df.loc[df["uso_banda"] == "0–0.3", "mora_grave"].mean()
uso_alto = df.loc[df["uso_banda"].isin(["0.9–1", ">1"]), "mora_grave"].mean(
edad_joven = df.loc[df["edad_banda"] == "≤30", "mora_grave"].mean()
edad_mayor = df.loc[df["edad_banda"] == "70+", "mora_grave"].mean()
q1 = df.loc[df["ingreso_grupo"] == "Q1 (bajo)", "mora_grave"].mean()
q5 = df.loc[df["ingreso_grupo"] == "Q5 (alto)", "mora_grave"].mean()

lines = []
lines.append("### Hallazgos clave")
lines.append("")
lines.append("1. **La hipótesis se sostiene:** el historial de atrasos domir
lines.append(f"   - `total_atrasos` IV={r.loc['total_atrasos', 'IV']:.3f} (A
lines.append(f"   - `atrasos_90` IV={r.loc['atrasos_90', 'IV']:.3f}; `atrasc
lines.append(f"   - Sin ningún atraso la mora es ~{t0:.1%}; con 3+ llega a ~
lines.append("")
lines.append(f"2. **El código de sistema 96/98 es una alarma roja:** mora ~{
lines.append("   No es ruido: es una de las señales más fuertes y debe entra
lines.append("")
lines.append(f"3. **El uso de crédito revolvente es el co-protagonista** (IV
lines.append(f"   Clientes en 0–0.3 tienen mora ~{uso_bajo:.1%}; en 0.9–1 /
lines.append("   En la práctica: alguien que ya quema su línea revolving es
lines.append("")
lines.append(f"4. **La edad protege hacia arriba:** ≤30 ~{edad_joven:.1%} vs
lines.append("   No es la variable #1, pero aporta segmentación clara para p
lines.append("")
lines.append(f"5. **Ingreso ayuda, pero menos que atrasos/uso:** Q1 (bajo) ~
lines.append("   Quienes no reportan ingreso no son el grupo más riesgoso; s
lines.append("")
lines.append("6. **Multicolinealidad entre atrasos:** las tres columnas se s
lines.append("   Para `03_modelo` conviene `total_atrasos` + `codigo_atraso_
```

```
lines.append(f"   comparar logística vs. un modelo de árboles, y evaluar cor
print("\n".join(lines))
```

Hallazgos clave

1. **La hipótesis se sostiene:** el historial de atrasos domina el ranking.
   - `total_atrasos` IV=1.348 (AUC=0.768).
   - `atrasos_90` IV=0.827; `atrasos_30_59` IV=0.702.
   - Sin ningún atraso la mora es ~2.8%; con 3+ llega a ~44.6% (vs global 6.7%).

2. **El código de sistema 96/98 es una alarma roja:** mora ~54.6% (más de 8× el promedio).
   No es ruido: es una de las señales más fuertes y debe entrar al modelo como bandera.

3. **El uso de crédito revolvente es el co-protagonista** (IV=1.113).
   Clientes en 0–0.3 tienen mora ~2.2%; en 0.9–1 / >1 sube a ~22.4%.
   En la práctica: alguien que ya quema su línea revolving es candidato a mora aunque aún no haya atrasado.

4. **La edad protege hacia arriba:** ≤30 ~11.6% vs 70+ ~2.3%.
   No es la variable #1, pero aporta segmentación clara para políticas (límites, precios).

5. **Ingreso ayuda, pero menos que atrasos/uso:** Q1 (bajo) ~9.1% vs Q5 (alto) ~4.5%.
   Quienes no reportan ingreso no son el grupo más riesgoso; sí obligan a tratar el `ratio_deuda` con cuidado.

6. **Multicolinealidad entre atrasos:** las tres columnas se solapan.
   Para `03_modelo` conviene `total_atrasos` + `codigo_atraso_especial` + `uso_credito_revolvente` (+ edad/ingreso),
   comparar logística vs. un modelo de árboles, y evaluar con AUC / matriz de confusión cuidando el desbalance (~6.7% mora).

In [17]:
```
rank_path = ROOT / "reports" / "ranking_iv_auc.csv"
ranking.to_csv(rank_path, index=False)
print(f"Ranking guardado en {rank_path.resolve()}")
print("Figuras guardadas:")
for p in sorted(FIG.glob("*.png")):
    print(f"  - {p}")
```

```
Ranking guardado en /workspace/gmsc/reports/ranking_iv_auc.csv
Figuras guardadas:
  - ../reports/figures/01_balance_objetivo.png
  - ../reports/figures/02_mora_por_atrasos.png
  - ../reports/figures/03_mora_codigo_especial.png
  - ../reports/figures/04_mora_por_edad.png
  - ../reports/figures/05_mora_por_ingreso.png
  - ../reports/figures/06_mora_por_uso_revolvente.png
  - ../reports/figures/07_mora_por_ratio_deuda.png
  - ../reports/figures/08_mora_lineas_hipotecas_deps.png
  - ../reports/figures/09_ranking_IV_AUC.png
  - ../reports/figures/10_correlacion_heatmap.png
```

Cierre

Siguiente paso	Qué haremos
`03_modelo.ipynb`	Regresión logística vs. modelo de árboles; AUC y matriz de confusión
Features prioritarias	`total_atrasos`, `codigo_atraso_especial`, `uso_credito_revolvente`, edad, ingreso (+ banderas)
Cuidado	Desbalance (~6.7%), colinealidad de atrasos, `ratio_deuda` dudoso sin ingreso